

Réseaux pour le cloud

Jour 2 — Acheminer : routage, NAT, DHCP et DNS

Hier vos paquets savaient qui chercher. Aujourd'hui ils apprennent le chemin.

Objectifs de la journée

À la fin de cette journée, vous serez capable de :

- Lire et expliquer une **table de routage** (`ip route`), y compris la **passerelle par défaut** et la règle du **préfixe le plus long**.
- Ajouter et justifier une **route statique**.
- Expliquer le **NAT** (source et destination) et pourquoi le cloud en dépend.
- Décrire le cycle **DHCP** (DORA) et la notion de bail.
- Expliquer la **résolution DNS** de bout en bout : hiérarchie, résolveur, serveurs autoritaires, cache et TTL.
- Diagnostiquer un problème DNS avec `dig` et lire les enregistrements A, AAAA, CNAME, MX, TXT, NS.

Échauffement d'abord : 5 calculs de sous-réseaux au tableau. Tous les matins.

Plan de la journée

0. **Échauffement subnetting** (15 min, ça ne se discute pas).
1. **Le routage** — tables, passerelle, routes statiques, `ip route`.
2. **Le NAT** — source, destination, et pourquoi le cloud vit dessus.
3. **DHCP** — obtenir une adresse automatiquement (DORA).
4. **DNS en profondeur** — hiérarchie, enregistrements, TTL, `dig`, zones.
5. Récap + quiz de fin de journée.

Deux exercices : tables de routage à compléter, diagnostic DNS avec `dig`.
Une démo fil rouge : la résolution DNS décortiquée en direct.



Échauffement – 5 calculs chrono (15 min)

Au brouillon, méthode posée, correction immédiate par un binôme au tableau :

1. Réseau et broadcast de `192.168.7.130/25` ?
2. Combien d'hôtes utilisables dans un `/21` ?
3. `10.33.98.14/27` : première et dernière adresse utilisable ?
4. Quel préfixe pour 200 hôtes ?
5. `172.16.10.10/20` et `172.16.20.10/20` : même sous-réseau ?

Rappel de la méthode : **octet intéressant** → **bloc = 256 – masque** → **multiple** ≤ → **suivant** – **1**. La ligne des poids en haut du brouillon : `128 64 32 16 8 4 2 1`.

1. Le routage

Comment un paquet trouve son chemin

La décision de base : voisin ou passerelle ?

Avant chaque envoi, votre machine fait **un seul calcul** (celui d'hier) :

- destination **dans mon sous-réseau** (même résultat adresse ET masque) → livraison **directe** : ARP + trame vers la MAC du destinataire ;
- destination **ailleurs** → trame vers la MAC de **la passerelle** (le routeur), qui se débrouille pour la suite.

```
10.0.1.10/24 ----> 10.0.1.20 : direct (même /24)
10.0.1.10/24 ----> 8.8.8.8   : via la passerelle 10.0.1.1
```

Le **routeur** répète ensuite la même question à chaque saut :

« vers où envoyer ce paquet ? » — la réponse est dans sa **table de routage**.

La table de routage

Une table de routage = une liste de règles « **pour telle destination, passe par là** » :

Destination	Via	Interface	Commentaire
10.0.1.0/24	– (directe)	eth0	mon propre sous-réseau
10.0.2.0/24	10.0.1.254	eth0	route statique vers un autre site
0.0.0.0/0	10.0.1.1	eth0	route par défaut : tout le reste

Règles de lecture :

1. On cherche **toutes** les routes qui correspondent à la destination.
2. La plus **spécifique** gagne : c'est le **longest prefix match** (un /24 bat un /16, qui bat /0).
3. **0.0.0.0/0** correspond à **tout** : c'est la route par défaut, celle qui mène à la **passerelle par défaut** (default gateway).

Lire `ip route` sur la VM

```
$ ip route
default via 192.168.122.1 dev enp1s0 proto dhcp metric 100
192.168.122.0/24 dev enp1s0 proto kernel scope link src 192.168.122.50
```

Traduction ligne à ligne :

- `default via 192.168.122.1` : la **route par défaut** passe par cette passerelle (`default` = `0.0.0.0/0`) ; `proto dhcp` : apprise par DHCP.
- `192.168.122.0/24 dev enp1s0 scope link` : mon sous-réseau, joignable **directement** par l'interface ; `src` : l'adresse que j'utilise en sortie.

Commandes associées :

```
ip route get 8.8.8.8           # quelle route serait choisie pour cette IP ?
sudo ip route add 10.99.0.0/24 via 192.168.122.254 # route statique
```


Un paquet traverse deux routeurs

```

PC-A                R1                R2                PC-B
10.0.1.10/24        10.0.1.1 | 10.0.12.1 10.0.12.2 | 10.0.2.1 10.0.2.20/24
|                   |                   |                   |
+--- LAN A -----+--- liaison /30 -----+--- LAN B -----+
  
```

```

Table PC-A : default via 10.0.1.1
Table R1   : 10.0.2.0/24 via 10.0.12.2 (route statique)
Table R2   : 10.0.1.0/24 via 10.0.12.1 (route statique)
Table PC-B : default via 10.0.2.1
  
```

IP source/destination : **inchangées** de bout en bout.
 MAC source/destination : **réécrites à chaque saut**.

Route statique ou routage dynamique ?

- **Route statique** : écrite à la main (`ip route add`). Simple, prévisible, parfaite pour un petit site... mais à maintenir soi-même.
- **Routage dynamique** : les routeurs s'échangent leurs routes via des protocoles (OSPF en interne, **BGP** entre opérateurs — le protocole qui fait tenir Internet). Hors programme : retenez juste les noms.

Et dans le cloud ?

- Chaque sous-réseau VPC est associé à une **table de routage que vous écrivez** : des routes statiques, exactement comme `ip route add`.
- La ligne `0.0.0.0/0 → igw-...` que vous verrez vendredi **est** une passerelle par défaut. Rien de nouveau : même concept, autre syntaxe.

Diagnostics : `tracert 8.8.8.8` montre les routeurs traversés (TTL décrémenté à chaque saut — le champ TTL du paquet IP, à ne pas confondre avec le TTL DNS).



Exercice 2-1 – Compléter des tables de routage (45 min)

Fichier : `exercices/04-cl-reseau/exercice-2-1-tables-de-routage.md`

Sur papier, une topologie à 2 routeurs et 3 réseaux :

- compléter les tables de routage de deux PC et deux routeurs ;
- appliquer le **longest prefix match** sur des cas ambigus ;
- suivre un paquet saut par saut (IP et MAC à chaque tronçon) ;
- diagnostiquer deux pannes classiques (passerelle hors sous-réseau, route retour manquante).

La panne « route aller OK, route retour absente » est **LE** grand classique – vous la recroiserez dans les VPC avec les peerings.

2. Le NAT

**Le tour de passe-passe qui fait tenir Internet
(et le cloud)**

Le problème : des adresses privées sur un Internet public

Vos machines utilisent des adresses **privées** (RFC 1918, vues hier) :

10.x, 172.16-31.x, 192.168.x. Or ces adresses **ne sont pas routées sur**

Internet : aucun opérateur ne sait où renvoyer une réponse vers 192.168.1.10.

La solution : le **NAT** (Network Address Translation). Un équipement de bordure (box, routeur, **NAT Gateway**) **réécrit les adresses** des paquets qui le traversent et tient un tableau des correspondances.

Deux grandes familles :

- **SNAT** (source NAT) : je **sors** vers Internet → on réécrit la **source** ;
- **DNAT** (destination NAT) : on me **contacte** depuis Internet → on réécrit la **destination** (redirection de port, load balancer).

SNAT : sortir en se faisant passer pour la passerelle

```

PC 192.168.1.10          NAT (public 203.0.113.5)          serveur 198.51.100.80
|                         |                               |
| src 192.168.1.10:40001 | src 203.0.113.5:61425             |
| dst 198.51.100.80:443  ---> | dst 198.51.100.80:443         ---> |
|                         |                               |
| <--- réponse inversée grâce à la table NAT :           |
| 203.0.113.5:61425 <=> 192.168.1.10:40001             |

```

Le NAT retient **(IP:port interne) ⇔ (IP:port public)** pour chaque connexion :
c'est le **PAT** (Port Address Translation) – des centaines de machines
partagent **une seule** adresse publique.

Conséquences du NAT — et pourquoi le cloud en dépend

Conséquences à bien comprendre :

- Une machine derrière un SNAT peut **sortir**, mais personne ne peut **l'initier depuis l'extérieur** : effet pare-feu de fait (asymétrie).
- Pour rendre un service interne joignable : **DNAT** (ex. « le port 443 public → 192.168.1.10:8000 ») ou un répartiteur de charge devant.

Dans le cloud, le NAT est **partout** :

- la **NAT Gateway** AWS = un SNAT managé : les sous-réseaux **privés** sortent (mises à jour, API) sans jamais être joignables depuis Internet ;
- l'« IP publique » d'une instance EC2 est en réalité un **DNAT 1:1** fait par l'infrastructure AWS : l'instance ne voit que son adresse privée ;
- l'Internet Gateway fait cette traduction 1:1 pour les sous-réseaux publics.

Vendredi, la question « pourquoi mon instance privée peut faire `apt update` mais n'est pas joignable ? » aura une réponse en un mot : **SNAT**.

3. DHCP

Obtenir une adresse sans lever le petit doigt

DHCP : le cycle DORA

DHCP (Dynamic Host Configuration Protocol) distribue automatiquement :
adresse IP + masque, passerelle par défaut, serveurs DNS, durée du **bail**.

Le dialogue en 4 temps – **DORA** :

Client	Serveur DHCP
-- 1. DISCOVER (broadcast) ----->	« un serveur DHCP ici ? »
<-- 2. OFFER -----	« je te propose 192.168.1.57 »
-- 3. REQUEST (broadcast) ----->	« je prends 192.168.1.57 »
<-- 4. ACK -----	« accordé, bail de 24 h »

- Le **bail** (lease) a une durée : le client **renouvelle** à mi-bail.
- Pas de réponse DHCP ? La machine s'auto-attribue une adresse **169.254.x.x** (APIPA) – la voir, c'est déjà un diagnostic.

DHCP côté pratique et côté cloud

Sur la VM Ubuntu :

```
ip a                                # "dynamic" = adresse obtenue par DHCP
networkctl status enp1s0            # bail, serveur DHCP, DNS reçus
sudo dhclient -v enp1s0            # renégocier (verbeux : on voit DORA)
```

Dans le cloud :

- **vous ne configurerez jamais de serveur DHCP** : le VPC en fournit un (chez AWS, c'est lui qui répond sur chaque sous-réseau) ;
- les instances EC2 reçoivent leur adresse privée **par DHCP** dans la plage du sous-réseau que **vous** avez calculée — votre plan d'adressage d'hier est servi par le DHCP d'AWS ;
- les « DHCP option sets » AWS permettent de pousser vos propres DNS.

Moralité : DHCP reste, seul l'opérateur du serveur change.

4. DNS en profondeur

L'annuaire distribué le plus critique du monde

Le problème et l'idée

Les humains retiennent `stockline.example.com`, les paquets IP exigent `203.0.113.10`. Le **DNS** (Domain Name System) fait la traduction – et bien plus (messagerie, vérification de domaine, répartition de charge). Pourquoi « en profondeur » aujourd'hui ?

- **Toute panne DNS ressemble à une panne réseau totale** : savoir le diagnostiquer vous distingue immédiatement.
- Dans le cloud : Route 53 (CL-AWS1 J7), les certificats TLS validés par DNS (ACM), les endpoints privés... tout repose sur ces mécanismes.

Un nom de domaine se lit **de droite à gauche** :

```
stockline.example.com.  
|           |           | +-- la racine "." (implicite)  
|           |           +---- TLD : com  
|           +----- domaine : example.com  
+----- sous-domaine / hôte
```

La hiérarchie : racine → TLD → autoritaire

```

      . (racine, 13 "lettres" de serveurs)
+-----+-----+
com      org      fr      <- TLD
|
example.com      <- serveurs AUTORITAIRES
|               du domaine (détiennent la zone)
stockline.example.com = 203.0.113.10
  
```

- Serveur **autoritaire** : détient la **zone** (le fichier des enregistrements) et fait foi pour son domaine.
- Personne ne connaît tout : chaque étage sait seulement **qui interroger à l'étage du dessous** (délégation via enregistrements NS).

Le résolveur : celui qui fait le travail

Votre machine ne parcourt pas la hiérarchie elle-même : elle demande à un **résolveur récursif** (fourni par DHCP : box, FAI, entreprise, ou public comme `8.8.8.8` / `1.1.1.1`).

```
client --(1) stockline.example.com ?--> RÉSOLVEUR
      | (2) demande à la racine  -> "vois com"
      | (3) demande à com       -> "vois ns1.example.com"
      | (4) demande à ns1       -> "203.0.113.10"
client <--(5) 203.0.113.10 ----- RÉSOLVEUR (et met en CACHE)
```

- Le client fait une requête **récursive** (« débrouille-toi ») ; le résolveur fait des requêtes **itératives** (étape par étape).
- Le **cache** évite de refaire le chemin : chaque réponse est gardée pendant son **TTL**.
- Sur Ubuntu : `resolvectl status` montre le résolveur utilisé (souvent le cache local `systemd-resolved` sur 127.0.0.53).

Les enregistrements à connaître par cœur

Type	Rôle	Exemple
A	nom → adresse IPv4	app.example.com. 300 IN A 203.0.113.10
AAAA	nom → adresse IPv6	app.example.com. 300 IN AAAA 2001:db8::10
CNAME	alias vers un autre nom	www.example.com. IN CNAME app.example.com.
MX	serveur de messagerie (+ priorité)	example.com. IN MX 10 mail.example.com.
TXT	texte libre (SPF, vérifications)	example.com. IN TXT "v=spf1 ..."
NS	serveurs autoritaires (délégation)	example.com. IN NS ns1.example.com.

Règles importantes :

- un **CNAME ne coexiste avec rien d'autre** sur le même nom, et jamais à la racine du domaine (example.com lui-même) ;
- MX et NS pointent vers des **noms** (qui ont eux-mêmes des A/AAAA), pas des IP.

Le TTL : la mémoire du DNS

Chaque enregistrement porte un **TTL** (Time To Live) en secondes :
la durée pendant laquelle les résolveurs ont le droit de garder la réponse
en cache.

```
app.example.com. 300 IN A 203.0.113.10
                  ^ ^ ^
                  5 minutes de cache autorisé
```

Conséquences très concrètes :

- TTL long (86400 = 24 h) : moins de requêtes, mais un changement d'IP met **jusqu'à 24 h** à se propager (en réalité : à **expirer des caches**) ;
- avant une migration : on **baisse le TTL à l'avance** (300 s), on migre, on remonte le TTL ;
- « la propagation DNS », c'est un abus de langage : rien ne se propage, **les caches expirent**.

Dans `dig`, le TTL affiché **décroît** à chaque interrogation du cache :
un excellent indice pour savoir si la réponse vient du cache ou de l'autoritaire.

dig : l'outil de référence

```
dig app.example.com           # requête A via le résolveur par défaut
dig app.example.com AAAA      # autre type d'enregistrement
dig @8.8.8.8 app.example.com  # interroger CE serveur précis
dig +short app.example.com    # juste la réponse
dig +trace app.example.com    # refaire le chemin racine -> TLD -> autoritaire
dig -x 203.0.113.10          # résolution inverse (PTR)
```

Lire une réponse `dig` :

- `status: NOERROR` (ok) / `NXDOMAIN` (le nom n'existe pas) / `SERVFAIL` (le résolveur a échoué) ;
- `ANSWER SECTION` : les enregistrements, avec leur TTL ;
- `flags: aa` = réponse d'un serveur **autoritaire** ;
- dernière ligne `SERVER:` = **qui** a répondu (méfiance si ce n'est pas celui attendu).

La zone : le fichier qui fait foi

Une **zone** est l'ensemble des enregistrements d'un domaine, chez ses serveurs autoritaires. Extrait type :

```
$TTL 3600
example.com.      IN SOA  ns1.example.com. admin.example.com. (
                    2026070201 ; serial (version de la zone)
                    7200 3600 1209600 300 )
example.com.      IN NS   ns1.example.com.
example.com.      IN NS   ns2.example.com.
app.example.com.  IN A     203.0.113.10
www.example.com.  IN CNAME app.example.com.
example.com.      IN MX    10 mail.example.com.
```

- **SOA** : carte d'identité de la zone (serial incrémenté à chaque modification) ;
- chez AWS, une zone = une **hosted zone** Route 53 : mêmes enregistrements, interface différente. Vous éditez ces lignes via la console et Terraform.



Démo 2-1 — La résolution DNS

décortiquée avec dig

Fichier : `demos/04-cl-reseau/demo-2-1-dns-dig.md`

En direct sur la VM :

- `resolvectl status` : qui est mon résolveur ?
- `dig utopios.net` : lecture guidée de **chaque section** de la réponse ;
- le TTL qui **décroît** entre deux requêtes : preuve du cache ;
- `dig +trace` : le chemin complet racine → TLD → autoritaire ;
- `dig CNAME`, `dig MX`, `dig NS`, `dig TXT` sur un domaine réel ;
- comparaison `@8.8.8.8` vs `@1.1.1.1` vs résolveur local.

Ensuite, à vous : l'exercice 2-2 vous met dans la peau de l'astreinte.



Exercice 2-2 – Diagnostiquer avec dig (45 min)

Fichier : `exercices/04-cl-reseau/exercice-2-2-diagnostic-dns.md`

Deux parties :

1. **Manipulation réelle** sur la VM : requêtes A/CNAME/MX/NS/TXT sur des domaines publics, mise en évidence du cache par le TTL.
2. **Diagnostic sur dossier** : cinq sorties `dig` fournies (NXDOMAIN, SERVFAIL, CNAME cassé, TTL et migration ratée, délégation NS incohérente) – pour chacune : le symptôme, la cause probable, l'action corrective.

C'est le format « ticket d'incident » : symptôme → hypothèse → preuve → correctif. Exactement ce qu'on attendra de vous en poste.

Récap des points clés du jour

- Décision d'envoi : **même sous-réseau = direct, sinon passerelle** — c'est le calcul d'hier qui décide.
- Table de routage : la route **la plus spécifique** gagne (longest prefix match) ; `0.0.0.0/0` = route par défaut.
- **SNAT** = sortir en masquant sa source (NAT Gateway) ;
DNAT = être joignable (redirection). Le cloud vit sur les deux.
- **DHCP** : Discover, Offer, Request, Ack — bail renouvelé à mi-vie ;
`169.254.x.x` = pas de DHCP.
- **DNS** : résolveur récursif + serveurs autoritaires + **cache TTL** ;
A/AAAA/CNAME/MX/TXT/NS ; `dig`, `+trace`, `NXDOMAIN` vs `SERVFAIL`.
- La « propagation DNS » n'existe pas : **les caches expirent**.



Quiz — questions 1 et 2

Question 1

Une machine `10.0.1.10/24` veut joindre `10.0.3.50`. Envoie-t-elle la trame directement au destinataire ou à sa passerelle ? Justifiez par le calcul.

Question 2

Dans une table de routage contenant `10.0.0.0/8 via R1` et `10.0.2.0/24 via R2`, par où part un paquet pour `10.0.2.7` ? Comment s'appelle cette règle ?



Quiz — questions 3 et 4

Question 3

Que signifie la ligne `default via 192.168.1.1 dev eth0 proto dhcp` dans la sortie de `ip route` ? (trois informations attendues)

Question 4

Quelle forme de NAT permet à un sous-réseau privé de télécharger des mises à jour sur Internet sans être joignable de l'extérieur ? Quel service AWS l'implémente ?



Quiz — questions 5 et 6

Question 5

Citez, dans l'ordre et avec leur sens, les quatre messages de l'acronyme DORA.

Question 6

Votre machine affiche l'adresse `169.254.37.201`. Qu'en déduisez-vous ?



Quiz — questions 7 et 8

Question 7

Quelle est la différence entre un **résolveur récursif** et un serveur **autoritaire** ? Lequel des deux votre machine interroge-t-elle directement ?

Question 8

Quel type d'enregistrement DNS crée un alias d'un nom vers un autre nom ?
Citez une restriction importante de ce type.



Quiz — questions 9 et 10

Question 9

Vous devez changer l'adresse IP d'un site samedi. Que faites-vous **plusieurs jours avant** avec le TTL de l'enregistrement A, et pourquoi ?

Question 10

`dig` renvoie `status: NXDOMAIN` pour `www.example.com`.

Que signifie ce statut, et quelle est la cause la plus probable ici ?

À demain !

Jour 3 — Transporter : TCP, HTTP, TLS, load balancing, pare-feu, VPN

Vos paquets savent s'adresser et s'acheminer. Demain, on monte dans les couches :
comment TCP fiabilise, comment HTTP dialogue, comment TLS chiffre — et comment
un load balancer et un pare-feu décident de qui passe. Journée dense : dormez bien.